SmartPark Technical Documentation

This document provides a comprehensive technical deep dive into the architecture, business logic, and operational flow of the Smart Parking Slot Manager (PSM). Designed for developers and system architects, it covers everything from folder structure to advanced enterprise features.

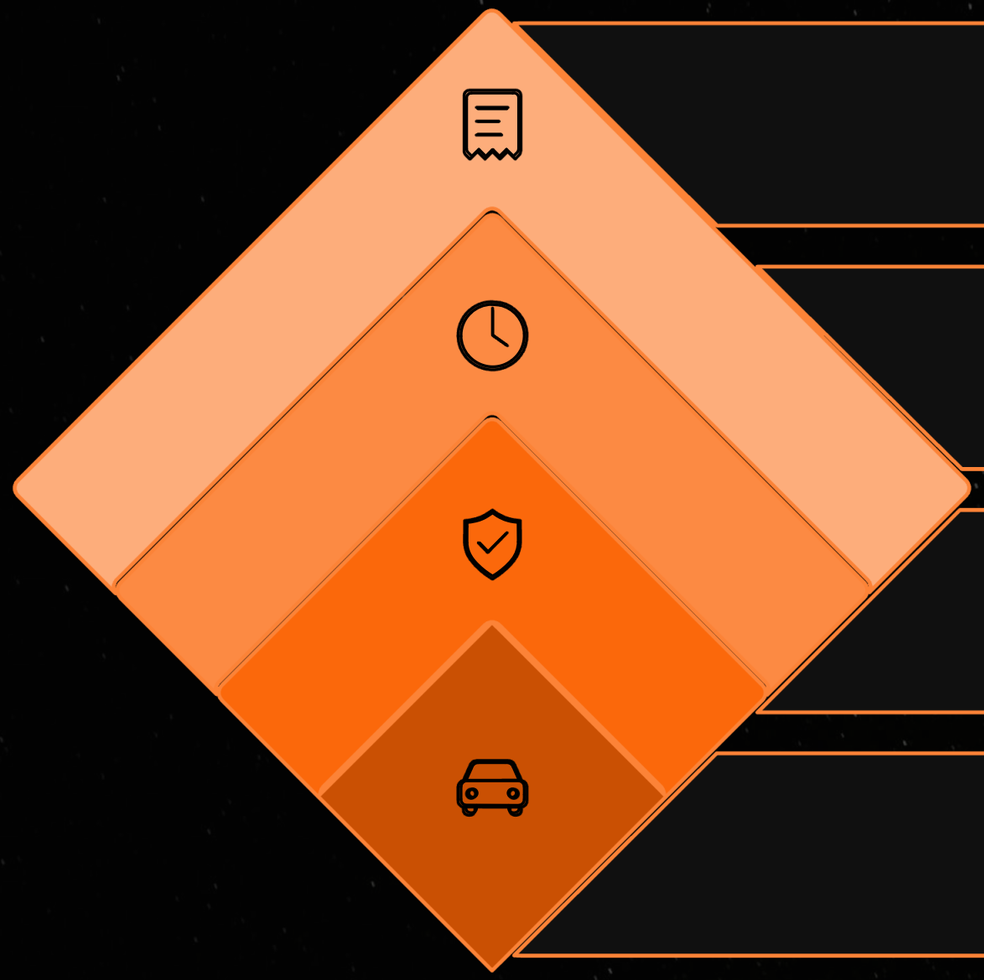
Project Folder Structure

The system follows a decoupled, microservice-ready architecture with clear separation of concerns between core management and payment processing.

```
SmartPark-PSM/
├── parking_manager/ # Core Management Service (Port 5000)
│ ├── app.py # Main Flask Backend (API & Logic)
│ ├── data/
│ │ └── parking_data.json # Persistent Slot & Transaction Data
│ └── static/
│ ├── logo.png # Branded Asset
│ ├── app.js # Frontend Logic, Timer, & Theme Engine
│ ├── index.html # Main Dashboard Template
│ ├── style.css # Global Theme & Print Styles
│ ├── parking_3d.html # Virtual Twin View
│ └── parking_3d.js # Three.js Logic
├── payment_gateway/ # Secure Payment Service (Port 5001)
│ ├── payment_app.py # Independent Backend
│ └── static/
│ ├── payment.html # Secure Pay Template
│ └── payment.js # Transaction Logic
└── README.md # Project Overview
```

User Flow Diagram

This diagram illustrates the complete lifecycle of a vehicle session from entry detection to slot release, including security checks and payment processing.



Complete & Release

Receipt then free slot

Session & Payment

Live timer, checkout/payment

Blacklist Check

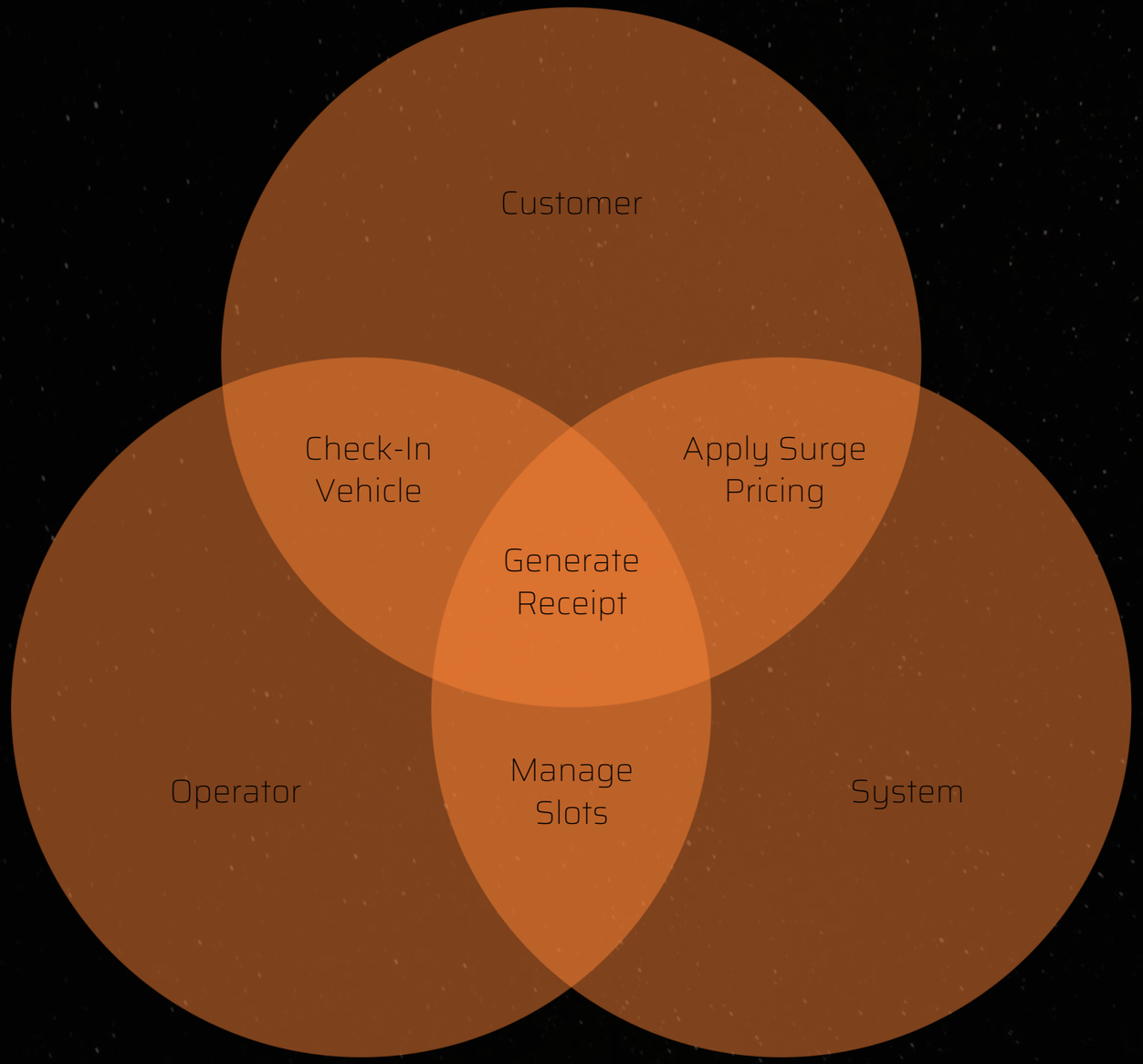
Branch to deny or allow

Vehicle Entry

Detect vehicle and record

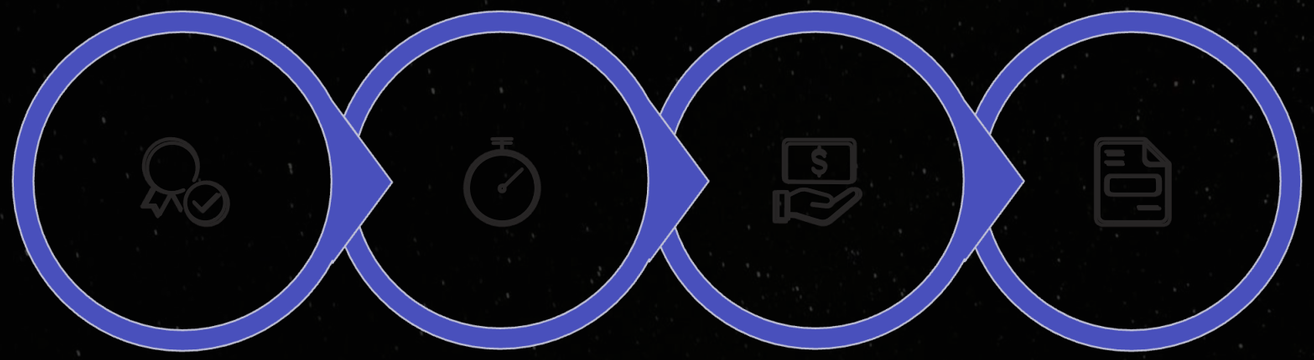
Use Case Diagram

Interaction mapping between system users (Operator and Customer) and core functional modules, showing actor responsibilities and system automation.



Smart Billing & GST Receipt Engine

SmartPark handles the full payment lifecycle — from slot entry to GST-compliant invoice — with zero manual intervention.



Vehicle Entry → **Duration Tracking** → **Payment Gateway** → **GST Invoice**

The billing engine calculates base rate + surge multiplier, applies member discounts, deducts GST at 18%, and generates a downloadable PDF invoice — all within seconds of vehicle exit.

Payment Gateway Integration

Supports UPI, card, and net banking via Razorpay / Stripe SDK. Payment status is verified server-side before slot release.

GST-Compliant Invoice

Auto-generated PDF with GSTIN, HSN code, CGST/SGST split, vehicle number, and transaction ID for audit trail compliance.

Dashboard

SmartPark

Dashboard
Parking Slots
Reservations
Vehicles
Transactions
Alerts 15
Settings
3D View
Memberships

AVAILABLE 56 | OCCUPIED 19

Dashboard

Search slot ID or plate...

Shift: OPEN (₹ 95)

15 SmartPark Central

TOTAL SLOTS	AVAILABLE	OCCUPIED	RESERVED	MAINTENANCE
80	56	19	3	2

Live Occupancy

23.8%

Zone A	Zone B	Zone C	Zone D
25%	20%	35%	15%

Revenue (Last 7 Days)

PEAK HOUR
4 AM

MOST USED SLOT
A14

Recent Activity

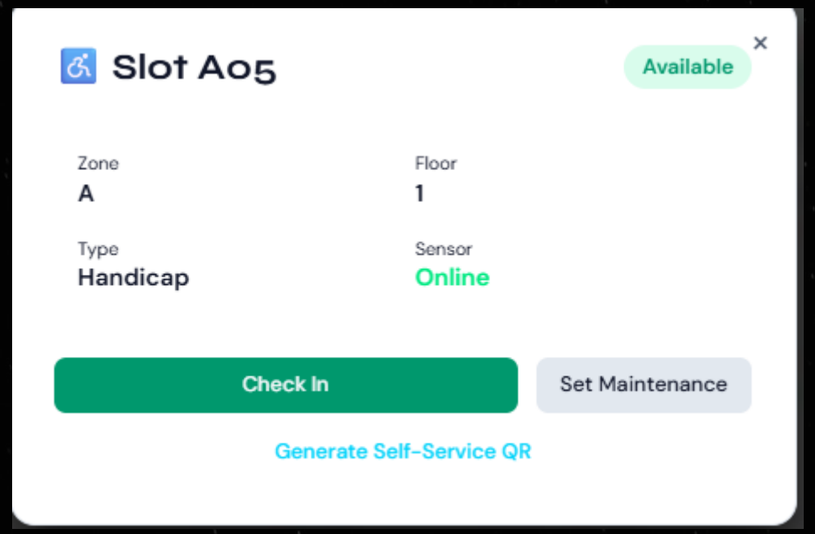
Checked out via wallet (Slot A05)
10:24 AM · ₹90

Checked in (Slot A05)
10:24 AM

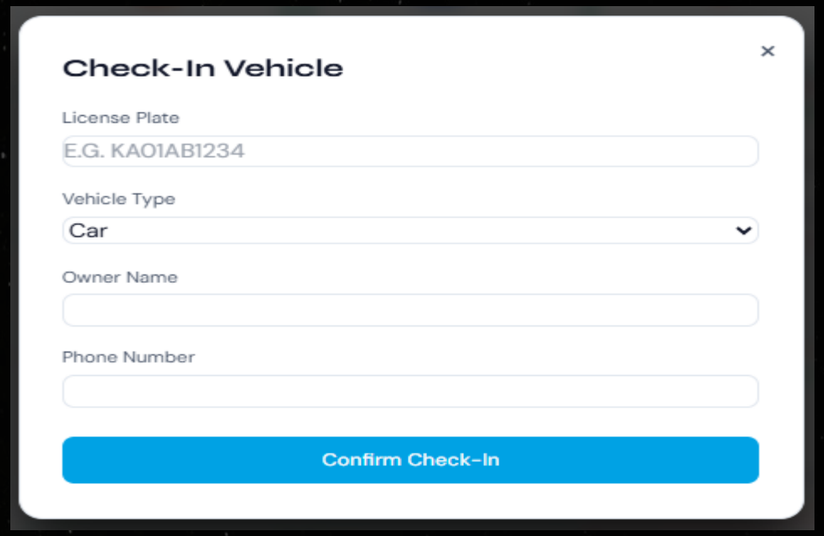
Checked out via wallet (Slot A05)
10:21 AM · ₹90

Created by Praveen

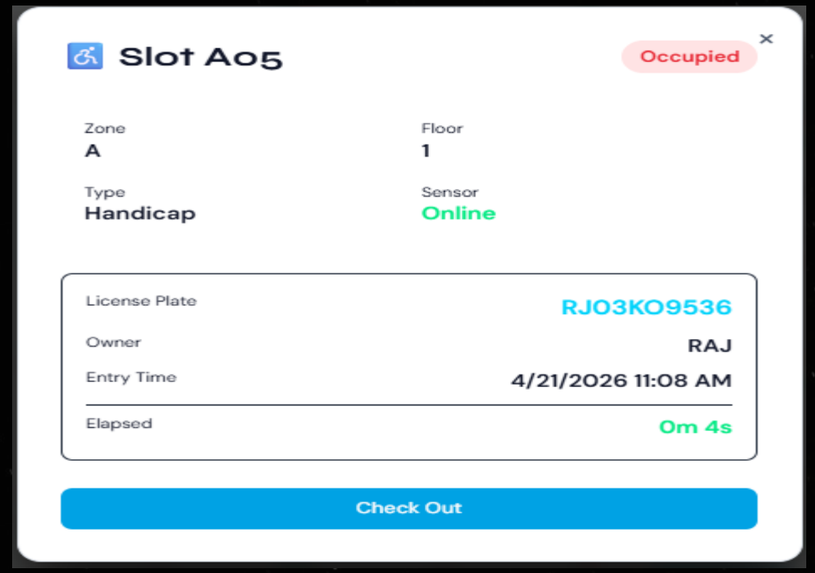
Check-In Popup



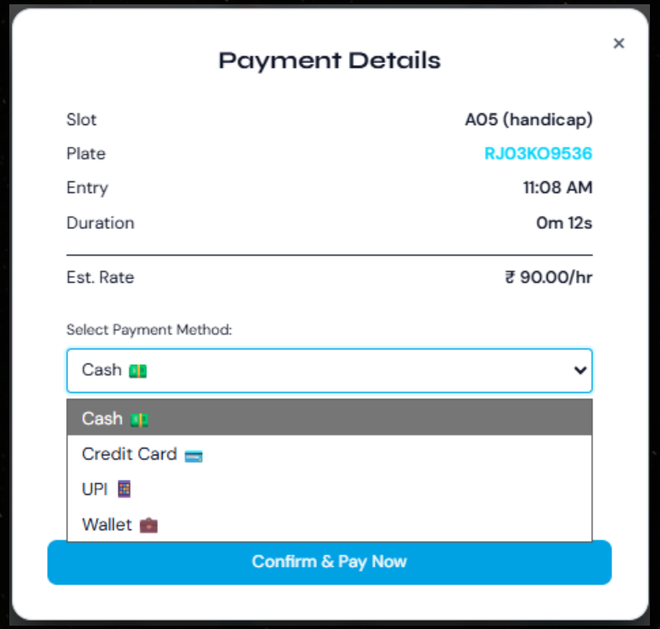
Vehicle Check-In System

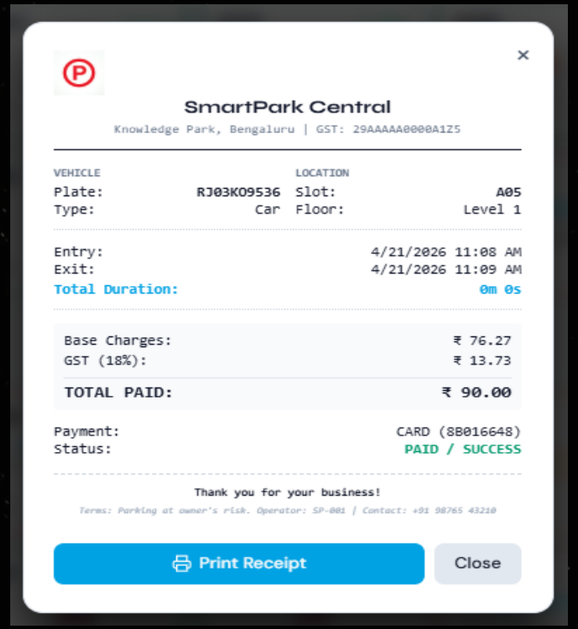


Vehicle Check-Out System

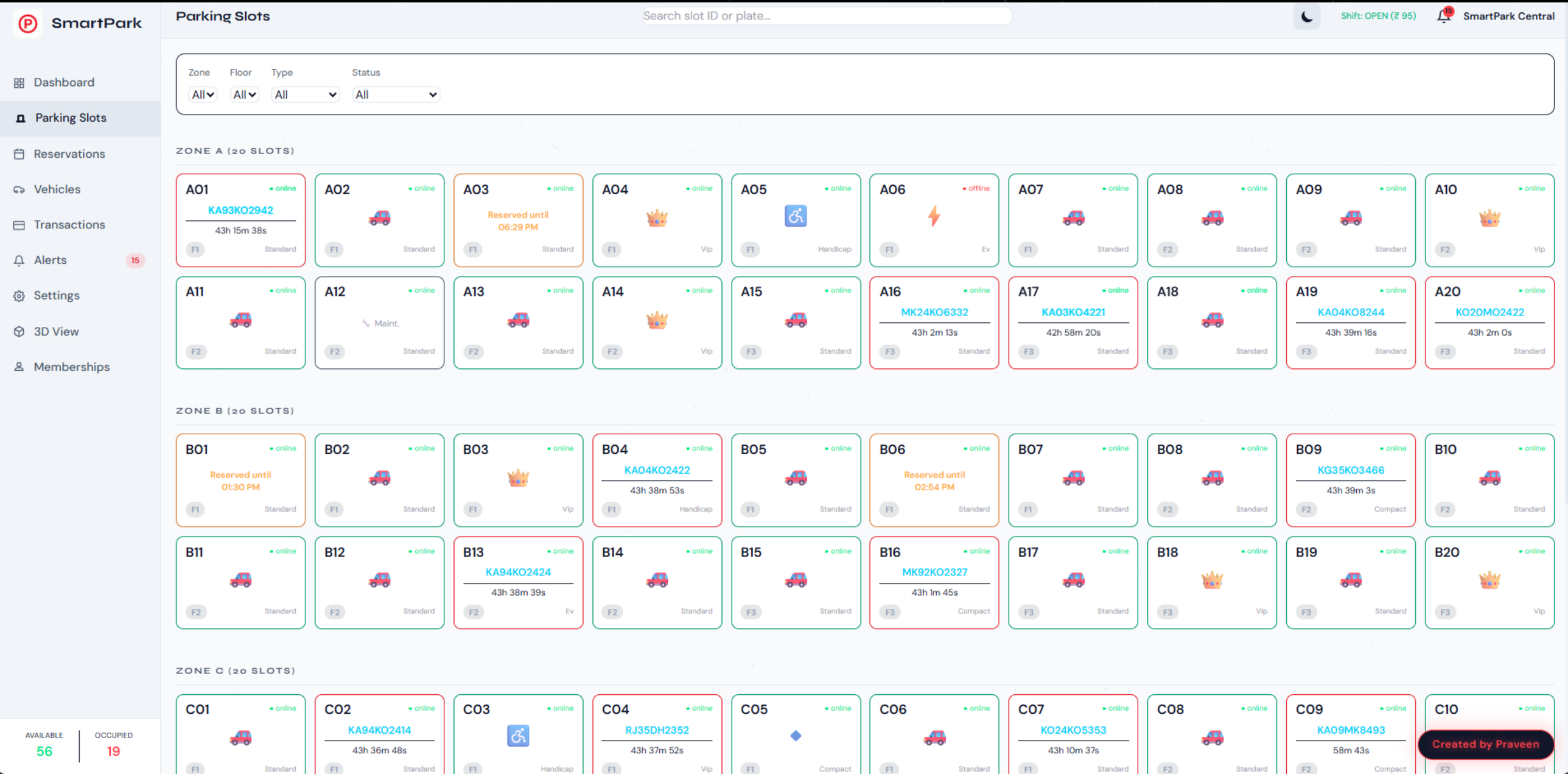


Parking Payment & Billing System





Parking Slot Overview & Control Panel :



Transactions History

SmartPark

Transactions

Search slot ID or plate...

Shift: OPEN (₹ 185)

15

SmartPark Central

- Dashboard
- Parking Slots
- Reservations
- Vehicles
- Transactions
- Alerts 15
- Settings
- 3D View
- Memberships

TOTAL REVENUE: ₹ 4450.93

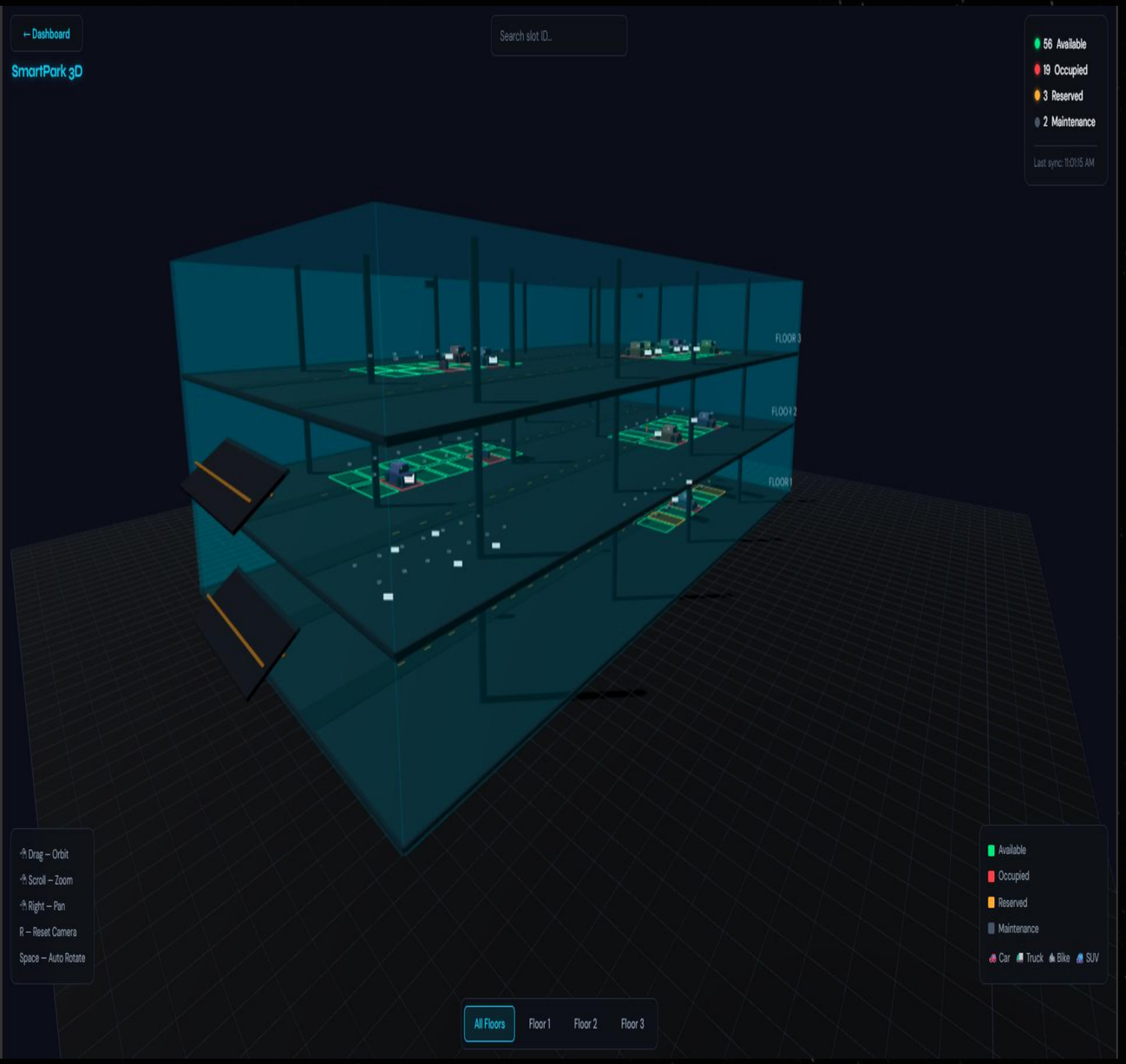
mm/dd/yyyy to mm/dd/yyyy Filter Export CSV

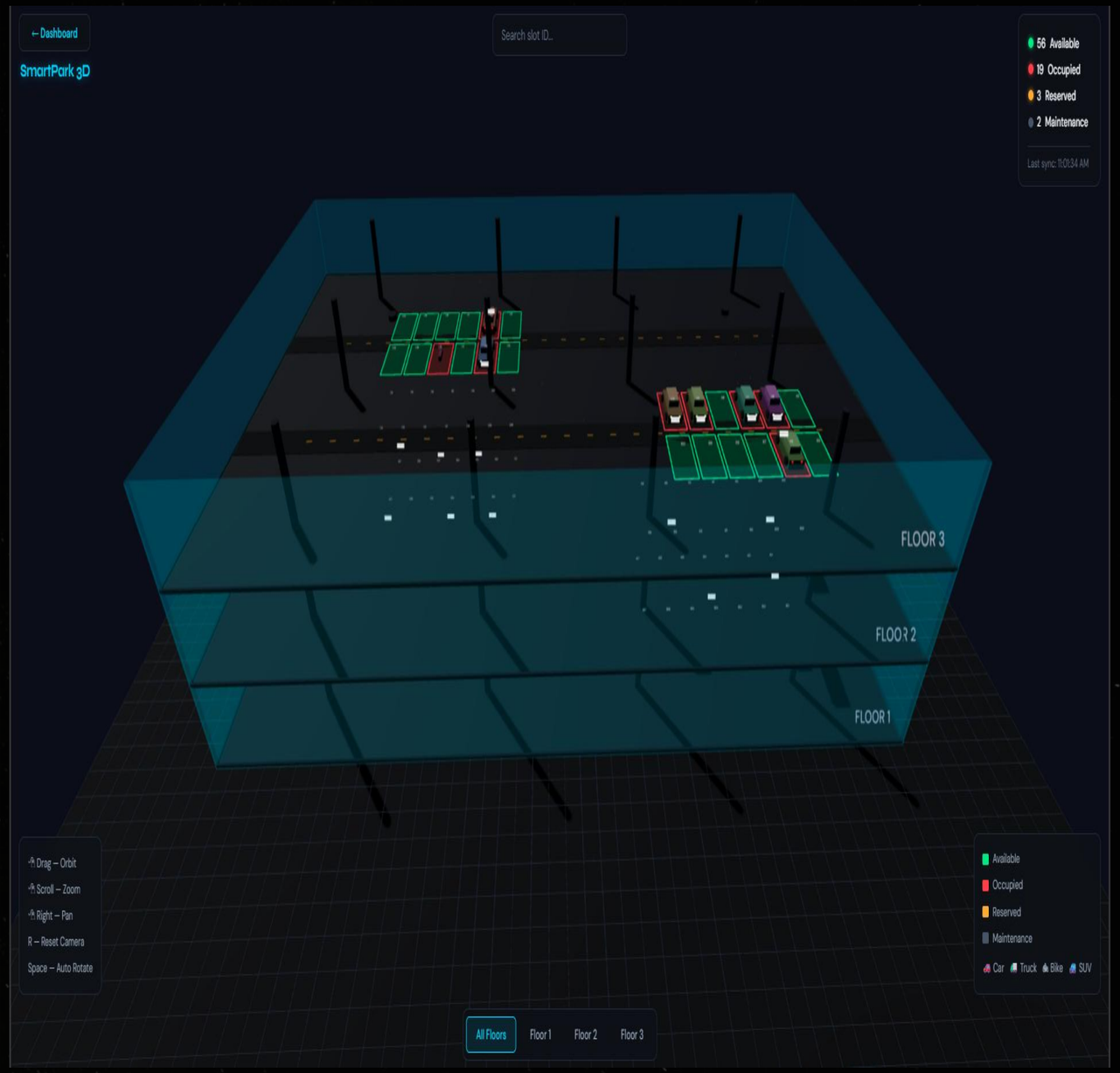
TX ID	SLOT	PLATE	OWNER	ENTRY	EXIT	DURATION	AMOUNT
8b016648	A05	RJ03KO9536	RAJ	4/21/2026 11:08 AM	4/21/2026 11:09 AM	0m 0s	₹ 90.00
64102ab7	A05	KA09KO8243	rajesh	4/21/2026 10:24 AM	4/21/2026 10:24 AM	0m 0s	₹ 90.00
9ec3b026	A05	KA08NK2421	raj	4/21/2026 10:21 AM	4/21/2026 10:21 AM	0m 0s	₹ 90.00
809d5a4d	A14	KA01KO4241	dhf	4/21/2026 10:17 AM	4/21/2026 10:17 AM	0m 0s	₹ 5.00
4cf58320	A14	KA09NK2422	raj	4/21/2026 10:16 AM	4/21/2026 10:17 AM	1m 0s	₹ 5.00
6d77e222	A14	KA04KO2412	raj	4/21/2026 10:14 AM	4/21/2026 10:14 AM	0m 0s	₹ 5.00
0cf7c85f	A14	KA09MK9403	MANISH	4/21/2026 10:13 AM	4/21/2026 10:13 AM	0m 0s	₹ 5.00
6a24bce5	D20	KA01MK9473	RAJ	4/21/2026 10:04 AM	4/21/2026 10:09 AM	5m 0s	₹ 90.00
3cde8b01	C11	KA03KO2421	sdgdg	4/19/2026 03:27 PM	4/21/2026 09:41 AM	42h 13m 0s	₹ 3800.93
9c40bde2	C12	KA01MK9983	LUCKY	4/21/2026 09:37 AM	4/21/2026 09:40 AM	3m 0s	₹ 90.00
daa1e15b	A01	KA01KO8239	twet	4/19/2026 03:25 PM	4/19/2026 03:49 PM	23m 0s	₹ 90.00
9c7cfe4d	C17	KS67JD3525	jdghdshs	4/19/2026 03:27 PM	4/19/2026 03:28 PM	1m 0s	₹ 90.00

AVAILABLE 56 | OCCUPIED 19

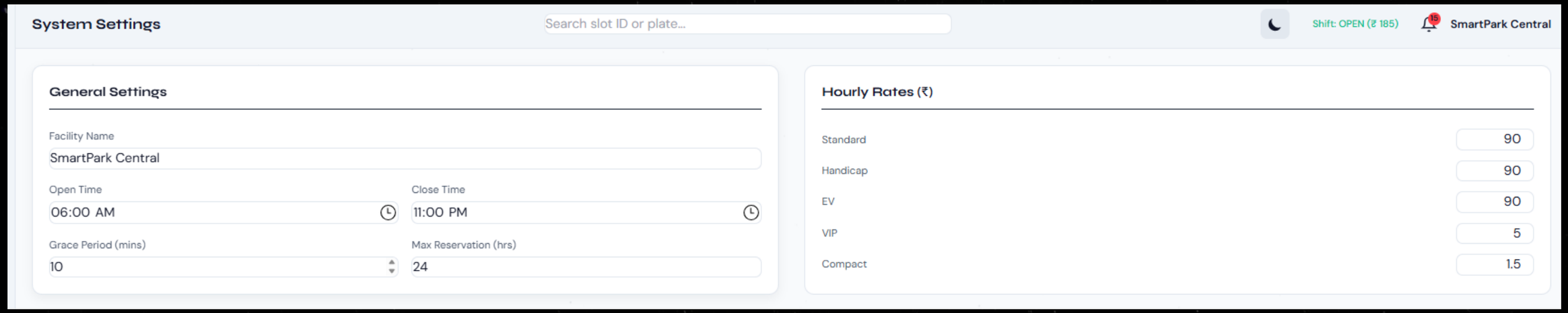
Created by Praveen

3D Digital Twin + Theme Engine :

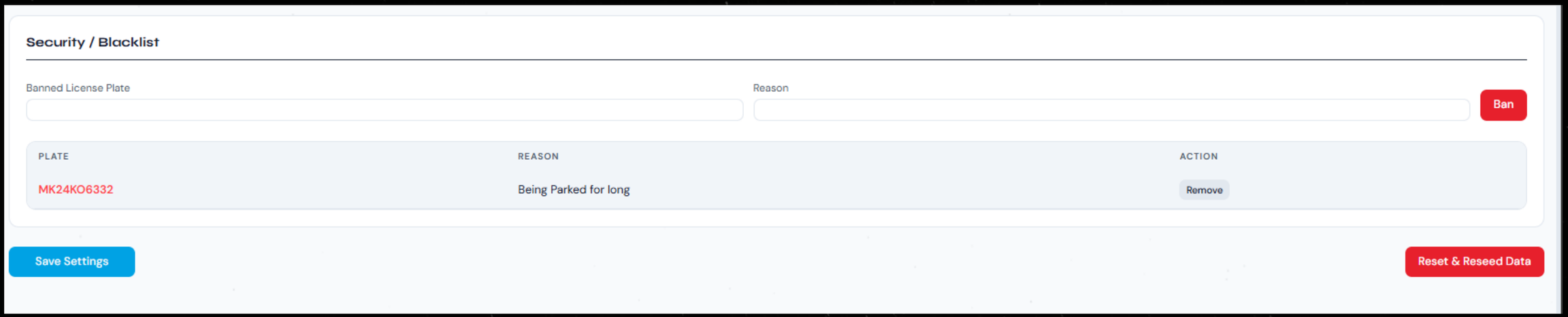




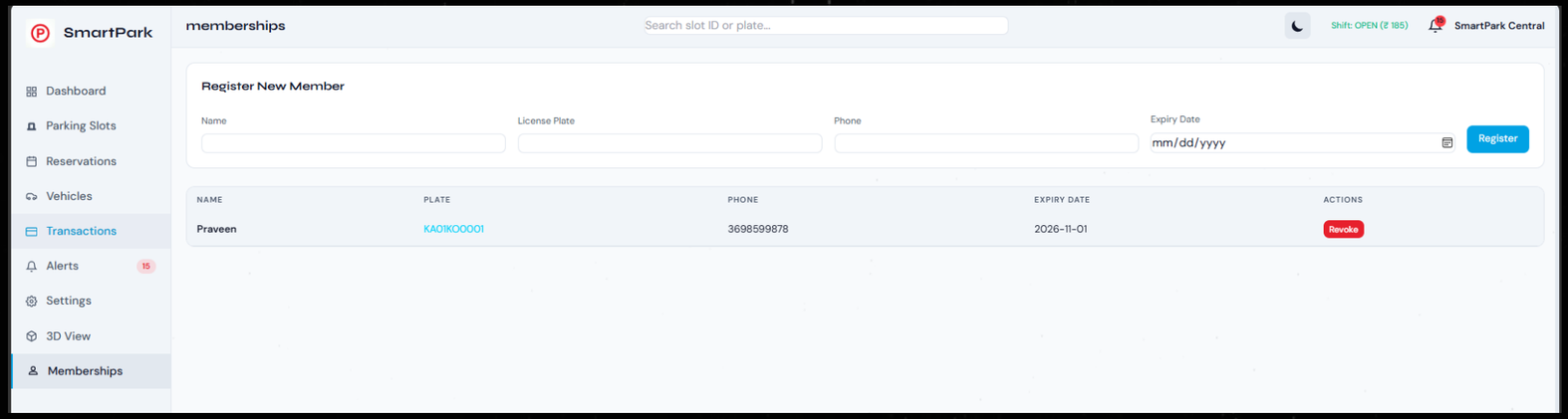
System Settings



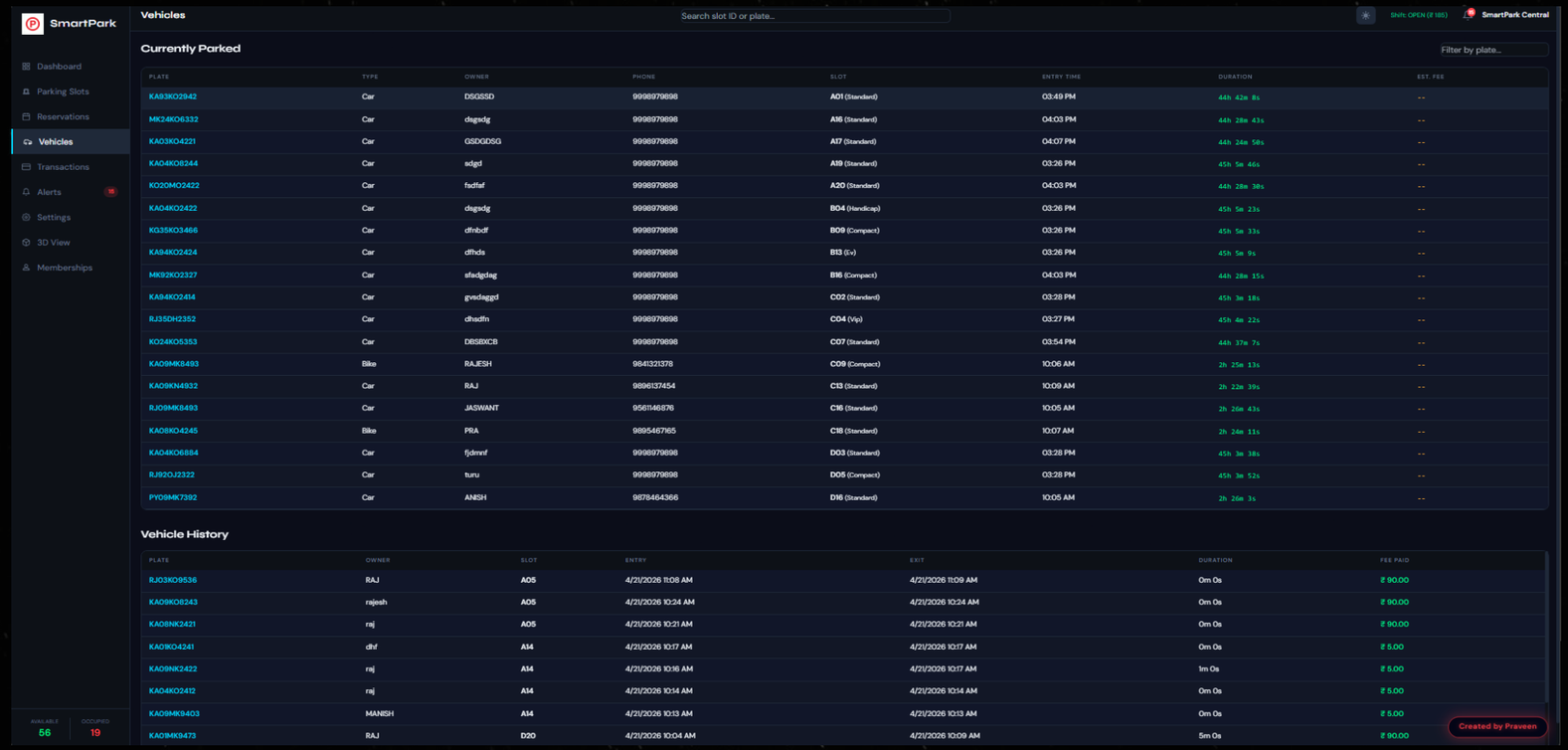
Blacklist Settings



Memberships



Vehicle History/Currently Parked



Overall Functional Flow

The system operates through five key phases that ensure real-time responsiveness, surge pricing adaptation, and seamless payment integration.

01

System Initialization

Upon launch, the system loads persistent slot data and system settings including hourly rates and facility information. The Theme Engine applies the user's saved preference or detects OS-level dark/light mode.

02

Live Monitoring Loop

A 1-second interval loop in app.js continuously updates live timers and occupancy metrics across all active slots, ensuring real-time dashboard accuracy.

03

Surge Detection

Monitors occupancy ratio continuously. When occupancy exceeds 85%, a surge_multiplier (1.5x) is automatically applied to calculate real-time fees for all active sessions.

04

Payment Handshake

The Checkout process passes a slot_id to the Payment Gateway via HTTP POST. Upon successful payment simulation, the Gateway sends a callback redirect back to the Manager with a transaction_id.

05

Receipt Rendering

The frontend uses cached appSettings to build a professional DOM-based receipt instantly, formatted for print using specialized CSS media queries that optimize for thermal printer output.

ARCHITECTURE

Tech Stack & System Architecture

SmartPark is built as a **Flask microservices architecture** with clearly separated concerns across frontend, backend, and data layers.

Flask

Core web framework & REST API

Three.js

WebGL 3D digital twin rendering

Python

Billing logic, PDF & surge engine

Razorpay / Stripe

Payment gateway SDK integration

Project Setup Guide

Prerequisites

- Python 3.8 or higher
- Flask web framework
- Flask-CORS extension
- QRCode library for receipts

Installation

```
pip install flask flask-cors qrcode
```

Running the Services

1. Start Manager: `python parking_manager/app.py`
2. Start Gateway: `python payment_gateway/payment_app.py`
3. Access Dashboard: `http://localhost:5000`

Key Code Logic & Feature Highlights

1

Adaptive Theme Engine

CSS Variable-based architecture with localStorage persistence. Automatically toggles light/dark mode class on body element and matches OS prefers-color-scheme on first visit. Smooth transitions applied to all theme properties.

2

Professional Receipt Generation

Global state caching of appSettings prevents exit delays. Synchronous DOM rendering uses cached facility data instead of fresh API calls during checkout. Financial math applies 18% GST breakdown with base amount and tax calculation.

3

Surge Pricing Implementation

Backend continuously monitors occupancy_ratio (occupied_slots/total_slots). When occupancy exceeds 85%, surge multiplier (1.5x) is applied to hourly rate for all active sessions. Real-time fee recalculation on every timer update.

4

Blacklist Security Logic

Security middleware intercepts every check-in request. Compares incoming license_plate against localized blacklist.json or persistent data store. Returns 403 Forbidden status on match, preventing slot occupation.

5

Shift Management & Reconciliation

Tracks all CASH transactions within timestamped session (shift_start to now). Shift Drawer automatically aggregates successful cash checkouts to provide "Expected Cash" value for operator during logout reconciliation.

Core Feature Implementation

Check-In & Slot Allocation

```
@app.route('/api/slots/<slot_id>/checkin',
methods=['PUT'])
def checkin(slot_id):
    data = request.json
    slot = find_slot(slot_id)
    slot['status'] = 'occupied'
    slot['vehicle'] = {
        'plate': data['plate'],
        'type': data['type'],
        'entry_time': datetime.now().isoformat()
    }
    save_data()
    return jsonify(slot)
```

Live Timer Updates

```
function updateLiveTimers() {
 const now = new Date();
 allSlots.filter(s => s.status === 'occupied')
 .forEach(slot => {
 const entry = new
Date(slot.vehicle.entry_time);
 const diff = Math.floor((now - entry) / 1000);
 const hours = Math.floor(diff / 3600);
 const mins = Math.floor((diff % 3600) / 60);
 document.getElementById(`timer-${slot.id}`)
 .innerText = `${hours}h ${mins}m`;
 });
}
setInterval(updateLiveTimers, 1000);
```

Blacklist Security Logic

```
async function loadBlacklist() {
  const res = await fetch(`${API}/blacklist`);
  const data = await res.json();
  document.getElementById('blacklist-table-
body').innerHTML =
    data.map(b => `
      <tr>
        <td class="text-red-500 font-
bold">${b.plate}</td>
        <td>${b.reason}</td>
      </tr>
    `).join('');
}
```

Secure payment logic

```
# ── TRANSACTION INIT ──
txn_id = f"TXN-{str(uuid.uuid4())[:10].upper()}"
txn = {
    "id": txn_id,
    "amount": float(body.get("amount")),
    "status": "pending",
    "created_at": datetime.utcnow().isoformat()
}
# ── FINAL RESPONSE ──
return {
    "txn_id": txn["id"],
    "status": txn["status"],
    "amount": txn["amount"]
}
```

Structured Data Storage Using JSON for Parking and Payment Systems

{} parking_data.json U

Parking_Slot_Manager-main > parking_manager > data > {} parking_data.json > ...


```json
{
  "slots": [
    {
      "id": "A01",
      "zone": "A",
      "number": 1,
      "type": "standard",
      "status": "occupied",
      "plate": "KA93KO2942",
      "owner": "DSGSSD",
      "phone": "9998979898",
      "vehicle_type": "Car",
      "entry_time": "2026-04-19T10:19:48.181048Z",
      "reserved_by": null,
      "reserved_until": null,
      "maintenance_reason": null,
      "floor": 1,
      "sensor_status": "online",
      "last_updated": "2026-04-19T10:19:48.181048Z"
    },
    {
      "id": "A02",
      "zone": "A",
      "number": 2,
      "type": "standard",
      "status": "available",
      "plate": null,
      "owner": null,
      "phone": null,
      "vehicle_type": null,
      "entry_time": null,
      "reserved_by": null,
      "reserved_until": null,
      "maintenance_reason": null,
      "floor": 1,
      "sensor_status": "online",
      "last_updated": "2026-04-21T10:24:47.995962Z"
    },
```


{} payment_data.json U

Parking_Slot_Manager-main > payment_gateway > data > {} payment_data.json > [ ] transactions > {} 2


```json
{
  "transactions": [
    {
      "id": "TXN-8A593C2A-9",
      "slot_id": "B07",
      "plate": "KA01AB1234",
      "amount": 8.26,
      "method": "wallet",
      "status": "success",
      "created_at": "2026-04-19T10:29:24.459633Z"
    },
    {
      "id": "TXN-AF6EF05E-0",
      "slot_id": "B07",
      "plate": "KA01AB1234",
      "amount": 8.26,
      "method": "wallet",
      "status": "success",
      "created_at": "2026-04-19T10:22:12.592887Z"
    },
    {
      "id": "TXN-29500EAA-7",
      "slot_id": "B07",
      "plate": "KA01AB1234",
      "amount": 8.26,
      "method": "wallet",
      "status": "success",
      "created_at": "2026-04-19T10:19:57.105329Z"
    },
```

DIFFERENTIATORS

Key Differentiators

What sets SmartPark apart from conventional parking software — features that deliver real operational and commercial value.

Pricing Engine

Demand-responsive pricing automatically raises rates during peak hours, increasing revenue per slot without manual intervention

Vehicle Blacklist

Operators can flag and block specific vehicle registration numbers. Blacklisted vehicles are denied entry at the gate automatically.

Membership & Priority Lanes

Monthly and annual membership plans with reserved spots, discounted rates, and fast-track entry for premium subscribers.

Deep Analytics

Hourly occupancy heatmaps, revenue trends, peak-hour analysis, and zone-level utilization reports — all exportable as CSV/PDF.

Thank You

SmartPark PSM — Redefining Urban Parking

From real-time dashboards and 3D visualization to smart billing and dynamic pricing — SmartPark delivers a complete, production-ready parking management experience.

🔗 Try the Live Demo

Run locally with `flask run` after cloning the repo.

📁 GitHub Repository

Full source code, setup guide, and documentation available on GitHub.

📬 Contact & Credits

Built by **Praveen Suthar**·
Stack: Flask, Three.js, Python